KADIR HAS UNIVERSITY

Faculty of Engineering And Natural Sciences

Department of Computer Engineering

CMPE 241 – Computer Programming II

TERM PHASE II PROJECT REPORT

Game Table Management System

Prepared by:

Cansın İsmail Bahçeci

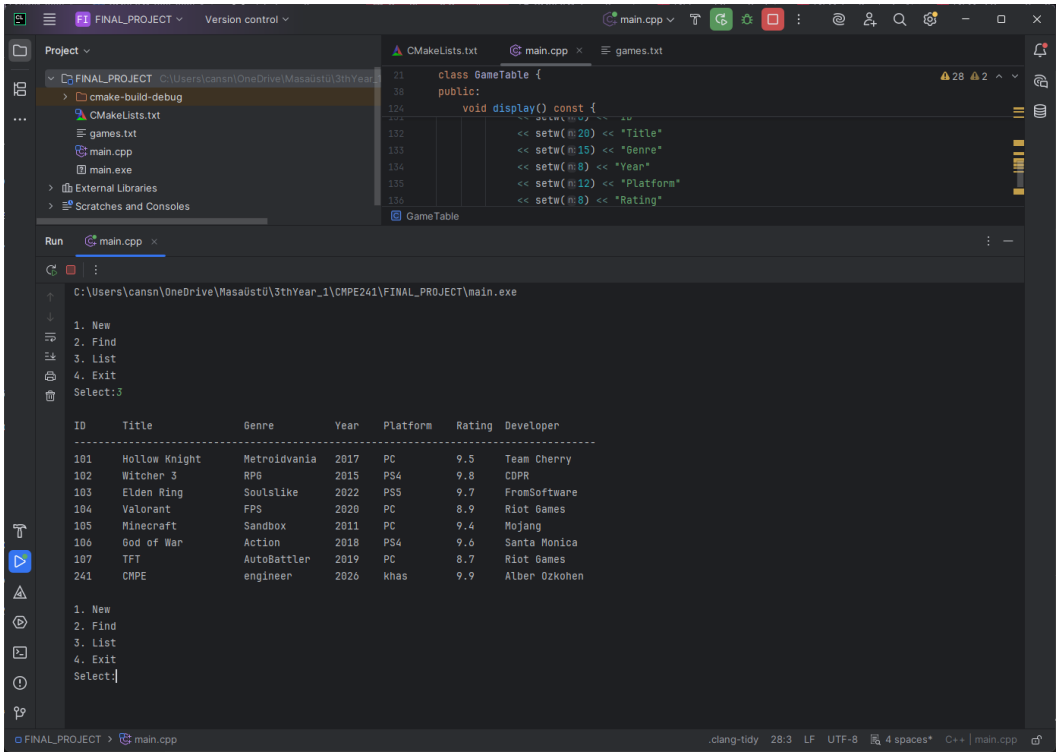
Student ID: 20221701030

Instructor: Prof. Albert Özkohen

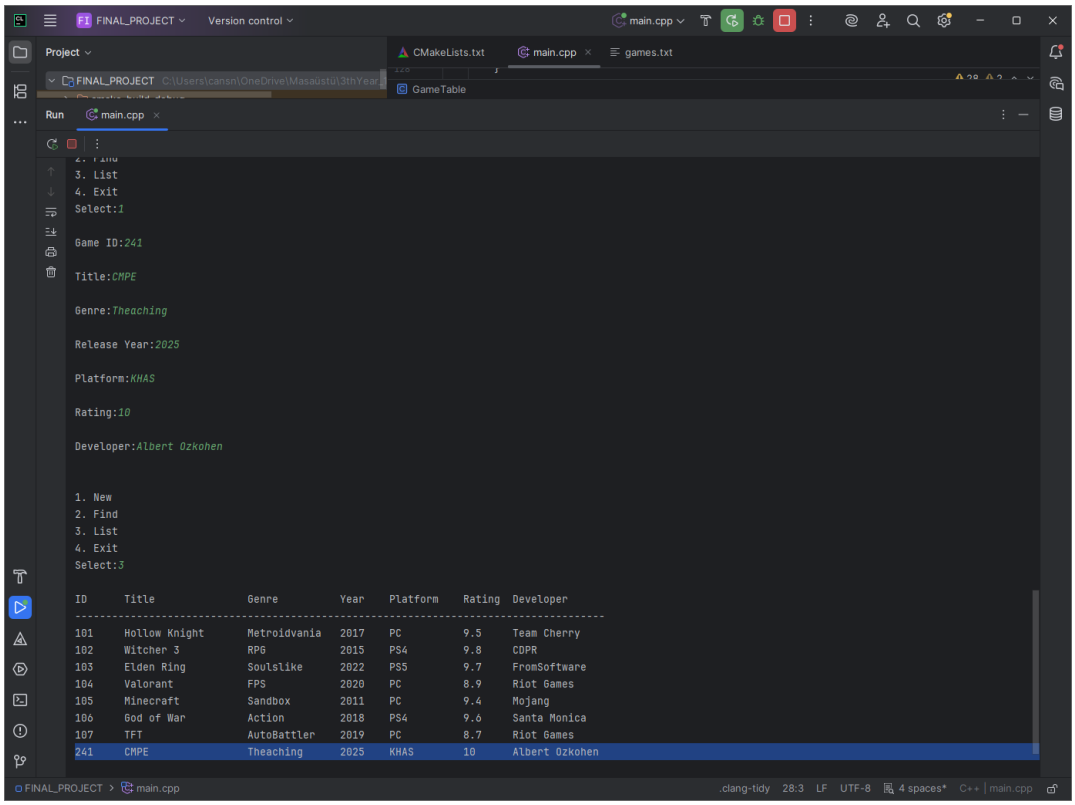
Date: 03.01.2026

Screen Shots Of The Program

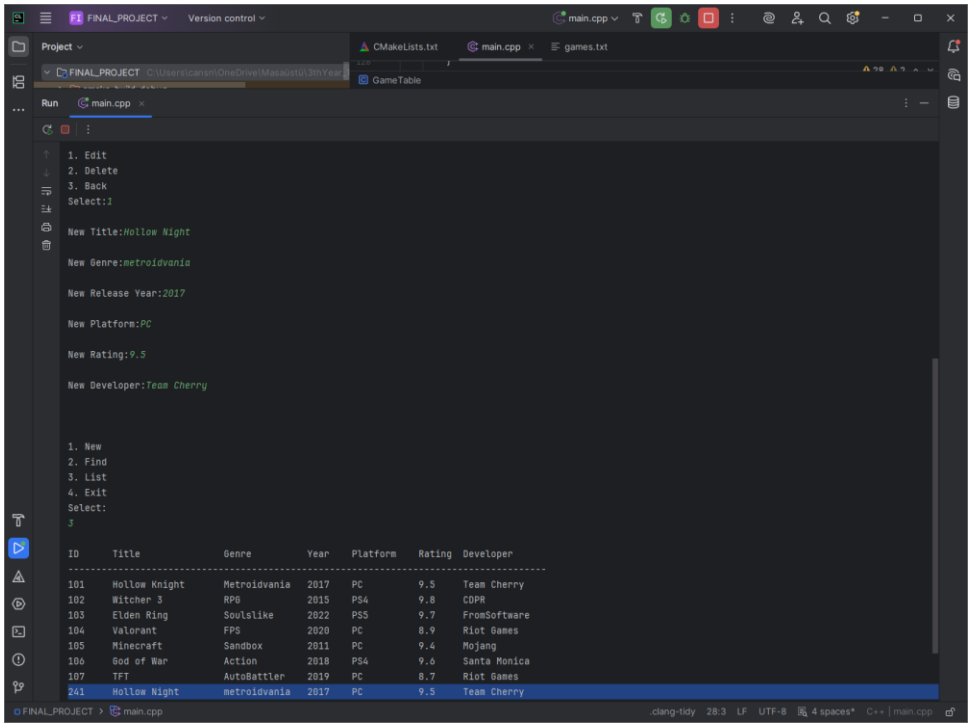
PRINT THE LIST



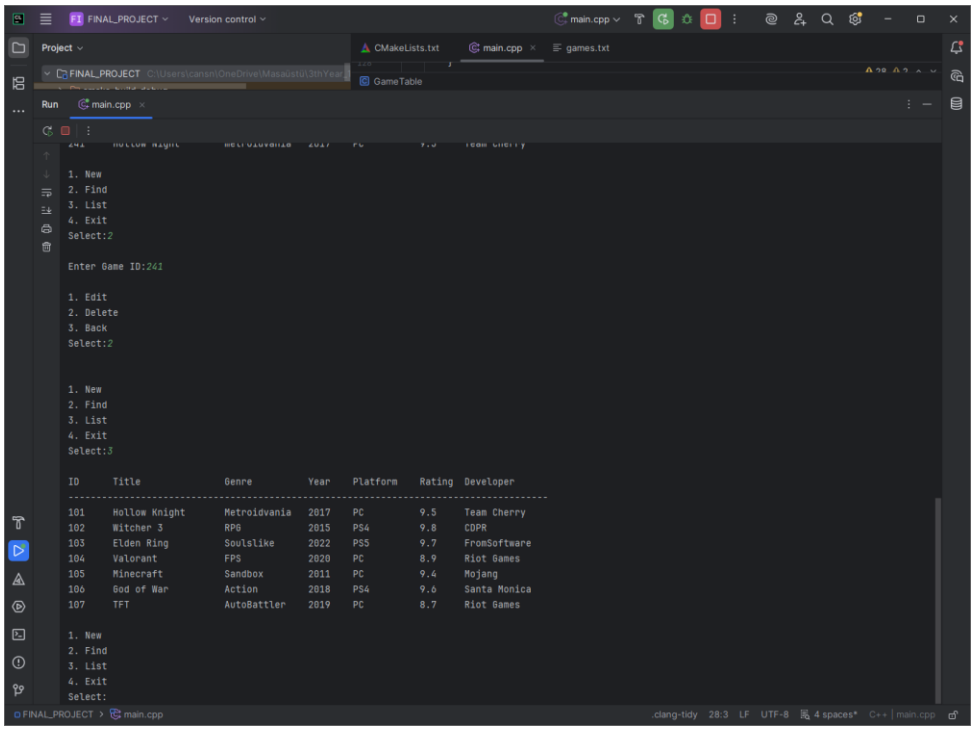
ENTER NEW GAME



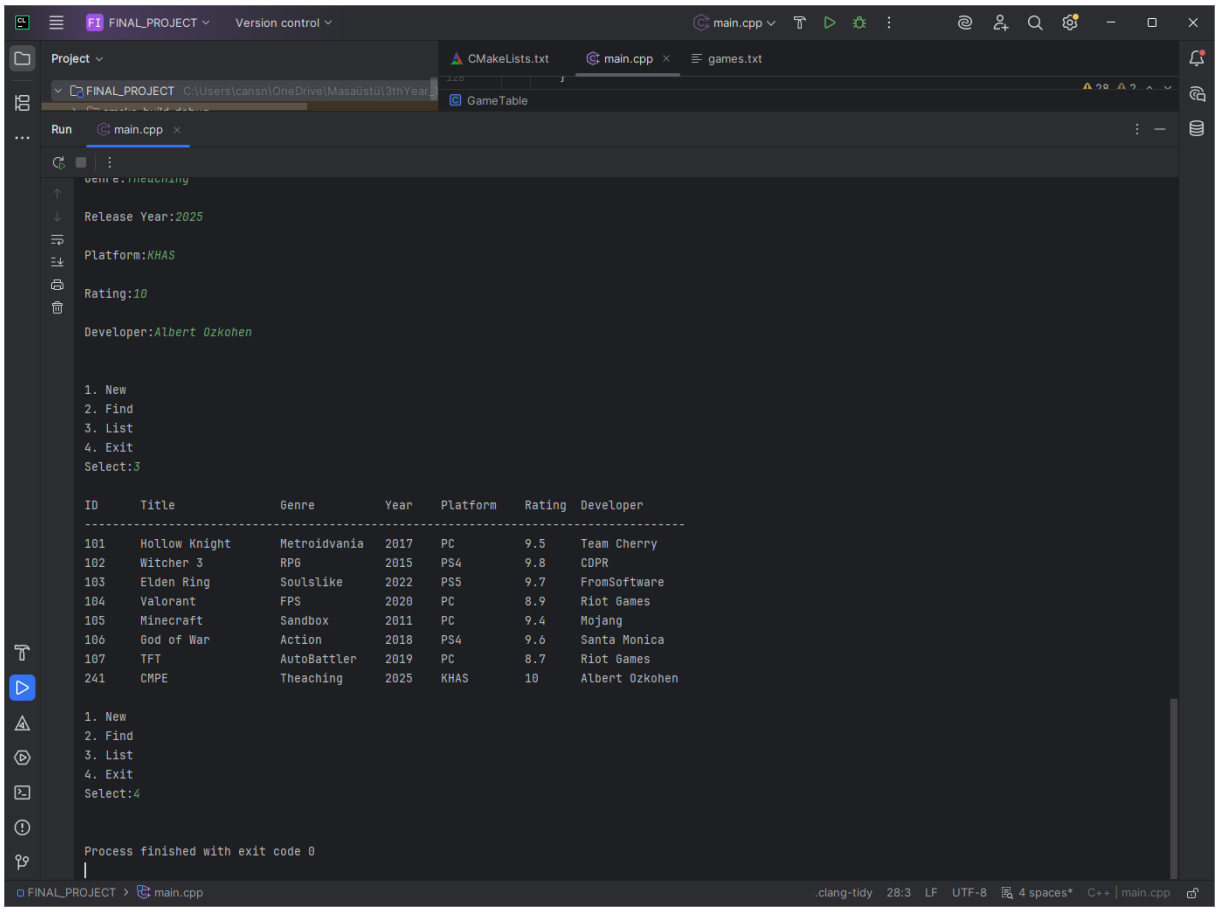
EDIT OPERATION



DELETE GAME



EXITING



THE CODE

```
#include <iostream>
#include <iomanip>
#include <string>
#include <fstream>

using namespace std;

class Game {
public:
   int id{};
   string title;
   string genre;
   int year{};
   string platform;
```

```
    float rating{};
    string developer;


};

class GameTable {
private:
    Game* games;
    int size;
    int capacity;
    const string fileName = "games.txt";

    void ensureCapacity() {
        if (size < capacity) return;
        capacity *= 2;
        Game* newArr = new Game[capacity];
        for (int i = 0; i < size; i++)
            newArr[i] = games[i];
        delete[] games;
        games = newArr;
    }

public:
    GameTable(int initialCap = 5) : size(0), capacity(initialCap) {
        games = new Game[capacity];
        loadFromFile();
    }

    ~GameTable() {
        saveToFile();
        delete[] games;
        games = nullptr;
    }

    GameTable(const GameTable&) = delete;
    GameTable& operator=(const GameTable&) = delete;


    void loadFromFile() {
        ifstream file(fileName);
        if (!file) return;

        Game g;
        while (file >> g.id) {
            file.ignore();
            getline(file, g.title);
            getline(file, g.genre);
            file >> g.year;
            file.ignore();
```

```
        getline(file, g.platform);
        file >> g.rating;
        file.ignore();
        getline(file, g.developer);
        ensureCapacity();
        games[size++] = g;
    }
    file.close();
}

void saveToFile() {
    ofstream file(fileName);
    for (int i = 0; i < size; i++) {
        file << games[i].id << "\n"
             << games[i].title << "\n"
             << games[i].genre << "\n"
             << games[i].year << "\n"
             << games[i].platform << "\n"
             << games[i].rating << "\n"
             << games[i].developer << "\n";
    }
    file.close();
}

void addGame(const Game& g) {
    ensureCapacity();
    games[size++] = g;
}

int findById(int id) const {
    for (int i = 0; i < size; i++)
        if (games[i].id == id)
            return i;
    return -1;
}

void editGame(int index) {
    cout << "New Title: ";
    getline(cin, games[index].title);
    cout << "New Genre: ";
    getline(cin, games[index].genre);
    cout << "New Release Year: ";
    cin >> games[index].year;
    cin.ignore();
    cout << "New Platform: ";
    getline(cin, games[index].platform);
    cout << "New Rating: ";
    cin >> games[index].rating;
    cin.ignore();
    cout << "New Developer: ";
```

```
        getline(cin, games[index].developer);
    }

    void deleteGame(int index) {
        for (int i = index; i < size - 1; i++)
            games[i] = games[i + 1];
        size--;
    }

    void display() const {
        if (size == 0) {
            cout << "Table is empty.\n";
            return;
        }

        cout << left
             << setw(8) << "ID"
             << setw(20) << "Title"
             << setw(15) << "Genre"
             << setw(8) << "Year"
             << setw(12) << "Platform"
             << setw(8) << "Rating"
             << setw(15) << "Developer"
             << "\n";

        cout << string(86, '-') << "\n";

        for (int i = 0; i < size; i++) {
            cout << left
                 << setw(8) << games[i].id
                 << setw(20) << games[i].title
                 << setw(15) << games[i].genre
                 << setw(8) << games[i].year
                 << setw(12) << games[i].platform
                 << setw(8) << games[i].rating
                 << setw(15) << games[i].developer
                 << "\n";
        }
    }
};

int readInt(const string& msg) {
    int value;
    while (true) {
        cout << msg;
        if (cin >> value) {
            cin.ignore(10000, '\n');
            return value;
        }
        cin.clear();
```

```
        cin.ignore(10000, '\n');
    }
}

float readFloat(const string& msg) {
    float value;
    while (true) {
        cout << msg;
        if (cin >> value) {
            cin.ignore(10000, '\n');
            return value;
        }
        cin.clear();
        cin.ignore(10000, '\n');
    }
}

int main() {
    GameTable table;

    while (true) {
        cout << "\n1. New\n2. Find\n3. List\n4. Exit\n";
        int choice = readInt("Select: ");

        if (choice == 1) {
            Game g;
            g.id = readInt("Game ID: ");
            cout << "Title: ";
            getline(cin, g.title);
            cout << "Genre: ";
            getline(cin, g.genre);
            g.year = readInt("Release Year: ");
            cout << "Platform: ";
            getline(cin, g.platform);
            g.rating = readFloat("Rating: ");
            cout << "Developer: ";
            getline(cin, g.developer);
            table.addGame(g);
        }
        else if (choice == 2) {
            int id = readInt("Enter Game ID: ");
            int index = table.findById(id);
            if (index == -1) {
                cout << "Game not found.\n";
                continue;
            }
            cout << "1. Edit\n2. Delete\n3. Back\n";
            int sub = readInt("Select: ");
            if (sub == 1) table.editGame(index);
            else if (sub == 2) table.deleteGame(index);
```

```
        }
        else if (choice == 3) {
            table.display();
        }
        else if (choice == 4) {
            break;
        }
    }
    return 0;
}
```